San Diego Movie Theater Ticketing System Software Requirements Specification

Version 1.0
July 10, 2026

Group 1
Anthony Alarcon (and Team Members)
Eric Phan, Ven Nguyen

Prepared for
CS 250 — Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.

San Diego State University

Revision History

Date	Version	Author	Comments
July 10, 2026	1.0	Anthony Alarcon et al.	Initial draft from client interviews and requirements elicitation

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date

	Anthony Alarcon Eric Phan Ven Nguyen	Software Engineer Software Engineer Software Engineer	July 10, 2026 July 10, 2026 July 10, 2026
	Dr. Gus Hanna	Instructor, CS 250	

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a complete, accurate and organized description of the requirements for the San Diego Movie Theater Ticketing System. It serves as the main structure for developers, testers, project managers, and other stakeholders during the development process. This document helps outline both the functional and non-functional requirements needed to create a secure, reliable, user-friendly, and a scaling ticketing system for a network of 20 movie theaters in San Diego County.

1.2 Scope

The SD Movie Theater Ticketing System is a web-based application that allows the customers to browse what movies are available and their showtimes, reserve seats, purchase up to 20 tickets in a single transaction, and receive either their tickets digitally or printed. Deluxe theaters will provide assigned seating, while regular theaters provide open seating. The system also offers the customers loyalty accounts, multiple payment options including credit/debit cards, PayPal, and Bitcoin, discounted pricing for eligible customers, post-purchase feedback, and access to movie ratings and reviews from sources such as Rotten Tomatoes and IMDb.

This system can easily be accessed through modern desktop and mobile web browsers, as well as kiosks that are located inside the theaters. It includes an administrative portal that enables theater staff to manage showtimes, process purchase overrides, and help resolve customer issues. To support business operations, the system keeps daily transaction logs, is designed to handle at least 1,000 customers that are operating it at the same time during peak hours, and uses security measures to generate unique, non-duplicable tickets that help prevent fraud and ticket scalping.

The initial release will not include native mobile applications, expanded locations outside San Diego County, subscription-based movie passes, or integration with third-party platforms.

1.3 Definitions, Acronyms, and Abbreviations

- **Actor:** A user or external system that interacts with the System in a particular role. • **Deluxe Theater:** Theater with assigned seating and 75 seats.
• **Loyalty Account:** Optional registered customer profile storing personal information,

payment methods, purchase history, and loyalty points.

- **Regular Theater:** Theater with open seating and 150 seats.
- **Showing:** A specific movie screening at a particular date, time, and theater.
- **SRS:** Software Requirements Specification.
- **UC:** Use Case.
- **Unique Ticket:** A cryptographically secure or NFT-style ticket that cannot be duplicated or resold fraudulently.

1.4 References

- Client Interview Notes – Theater Ticketing System Qs.rtf (July 2026)
- Elicited Requirements – theather-ticketing-reqs.txt and theather-ticketing-questions.txt
- Additional Requirements – Theater Ticketing Requirements.rtf
- Lecture Notes: Scenarios and Use Cases – CS 5150, Cornell University (William Y. Arms)
- CS 250 SRS Template – Instructor Gus Hanna, Ph.D.

1.5 Overview

This SRS is split up into several sections. Section 2 explains the product, its users, and any limitations. Section 3 covers the system requirements, including the interfaces, functional requirements, use cases, and non-functional requirements. The remaining sections such as classes, analysis diagrams, and change management will be completed in later assignments.

2. General Description

2.1 Product Perspective

The SD Movie Theater Ticketing System is a web-based application that works with external services for payment processing (credit/debit cards, PayPal, and Bitcoin), movie reviews (Rotten Tomatoes and IMDb), and the theater's existing database. Customers can access the system through any web browser or kiosks that are located in the theater. The system also manages user accounts, loyalty information, and ticket purchases while keeping data updated across all theaters in the SD County area.

2.2 Product Functions

The primary functions of the System include:

- Browsing movies, showtimes, and theater information
- Seat selection and reservation (assigned in deluxe, open in regular)
- Ticket purchase (1–20 tickets) with multiple payment options and discounts
- Digital ticket delivery (email/app) and physical printing at kiosks/box office
- Optional loyalty account registration, login, and management
- Display of critic reviews and scores
- Post-purchase customer feedback collection
- Administrative functions for staff (showtime management, overrides, reporting)
- Security and anti-scalping measures (unique tickets,

single-device login)

2.3 User Characteristics

Main users of the system are moviegoers of all ages and experience levels who can buy tickets online or at kiosks. Some customers may have the option to choose to create loyalty accounts to receive additional benefits. Theater staff and managers are also users of the system and have access to administrative features for managing showtimes, handling customer issues, and viewing reports. The system is designed to be clear and manageable requiring little to no training.

2.4 General Constraints

- Must be delivered as a responsive web application (no native mobile app required for v1) • Budget constraint: approximately $500,000 for initial development
- Prototype ready for user testing within 4 months
- Limited to the San Diego metropolitan area (Pacific Time Zone) and 20 existing theaters • Tickets valid only within the San Diego theater chain
- Minimal system downtime; updates must be performed with little to no service interruption

2.5 Assumptions and Dependencies

- Users have reliable internet access and a modern web browser.
- Payment gateways (credit card, PayPal, Bitcoin) are available and reliable.
- Existing theater showtimes and inventory databases can be integrated via API. • Third-party review APIs (Rotten Tomatoes / IMDB) remain accessible.
- Theater staff are available for in-person refund handling and kiosk support.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system will provide a user-friendly website that works well on desktop computers, mobile devices, and in-theater kiosks. The interface will support English, Spanish, and Swedish, and all prices will be displayed consistently using USD.

3.1.2 Hardware Interfaces

The system will work with standard web browsers on computers and mobile devices, as well as kiosks equipped with touchscreens, printers, and barcode/QR code scanners.

3.1.3 Software Interfaces

The system shall interface with:

- Payment gateway APIs (credit card processors, PayPal, Bitcoin)
- Movie review aggregation APIs (Rotten Tomatoes, IMDB)
- Existing partitioned theater showtime and inventory database

- Email/SMS gateway for digital ticket delivery

3.1.4 Communications Interfaces

The system will use secure HTTPS/TLS connections to protect all data sent between all users and the system. It will also constantly update seat availability and showtime information to ensure that everything is shown in real time so customers always see the most current information.

3.2 Functional Requirements

FR-01: Browse Showtimes and Movies

The system shall allow any user (guest or logged-in) to browse current and upcoming showtimes, movie details, and critic reviews/scores from Rotten Tomatoes and IMDB across all 20 San Diego theaters.

FR-02: Select Specific Seats

For deluxe theaters (75 seats), the system shall allow customers to view an interactive seat map and select specific seats. For regular theaters (150 seats), the system shall issue general admission tickets.

FR-03: Purchase Tickets (1–20 per Transaction)

Registered or guest users shall be able to purchase between 1 and 20 tickets for a single showing in one transaction. A 5-minute inactivity timer shall apply to the purchase alongside single session enforcement.

FR-04: Multiple Payment Methods

The system shall accept credit/debit cards, PayPal, and Bitcoin for ticket

purchases. **FR-05: Unique, Non-Replicable Tickets**

Every issued ticket shall be unique and non-replicable (implemented via secure digital token or NFT-style mechanism) to prevent scalping and fraudulent resale.

FR-06: Ticket Delivery Options

Customers shall receive tickets either digitally (via email or mobile app) or as printable/physical tickets at the box office or kiosk.

FR-07: Purchase Window

Tickets may be purchased up to 2 weeks in advance and up to 10 minutes after the scheduled showtime start.

FR-08: Optional Loyalty Accounts

Users may create optional loyalty accounts that store personal information, saved payment methods, purchase history, and accumulated loyalty points. Only one concurrent login per account is permitted.

FR-09: Discounts

The system shall support discounted pricing for students, military/veterans, and seniors (particularly on weekdays). Discount eligibility shall be verifiable at purchase time.

FR-10: Post-Purchase Feedback

After a successful purchase, the system shall prompt the customer to provide quick feedback using a simple smiley-face rating system.

FR-11: Administrative Functions

Authorized theater staff shall have access to an admin interface to set showtimes, manage theater configurations, override customer purchases when errors occur, and view reports.

FR-12: High-Demand Queueing

During periods of high demand for popular showings, the system shall implement a fair queueing mechanism to manage request volume without crashing.

FR-13: Daily Purchase Logging

The system shall maintain system-wide daily logs of all ticket purchases for auditing and reporting purposes.

FR-14: Multi-Language Support

The user interface shall be available in English, Spanish, and Swedish, with consistent pricing display (primarily USD).

3.3 Use Cases

Three primary use cases have been elaborated below following the format used in the course lecture materials (actor, flow of events, entry conditions). A use case diagram is provided at the end of this section.

UC-01: Purchase Tickets (Online)

Actor(s): Customer (guest or registered)
Purpose: Allow a customer to browse showtimes, select seats, purchase tickets, and receive confirmation for a specific movie showing.
Entry Conditions: Customer has internet access and a supported browser. The desired showing has available seats and is within the purchase window (up to 2 weeks in advance or 10 minutes after start). **Main Success Scenario (Flow of Events):**

1. Customer navigates to the SDTheaterTickets website or kiosk and browses available movies and showtimes.
2. Customer selects a specific showing (theater, date, time) and views the seat map (assigned seats for deluxe theaters).
3. Customer selects desired seats and enters the number of tickets (1–20).
4. System applies any applicable discounts (student, military, senior) if the customer provides valid credentials.

5. Customer reviews the order summary (tickets, seats, price, taxes) and proceeds to checkout. 6. Customer selects payment method (credit card, PayPal, or Bitcoin) and enters payment details. 7. System processes payment and, upon success, generates unique ticket(s).
8. Customer receives digital ticket(s) via email/app and/or prints physical ticket at kiosk. 9. System prompts customer for quick smiley-face feedback.
10. Customer logs out or continues browsing.

Extensions / Alternative Flows:

- Payment fails → System displays error and offers alternative payment method or cancellation. • Selected seat becomes unavailable during checkout → System notifies customer and suggests closest available seats.
- 5-minute timer expires → System releases held seats and returns customer to browse.
- Single-tab Enforcement → System ensures that only one browser instance per device can be opened, displays error until a session is terminated.

UC-02: Create and Manage Loyalty Account

Actor(s): Guest Customer → Registered Customer
Purpose: Allow a customer to create an optional loyalty account to store preferences, payment methods, history, and earn points.
Entry Conditions: Customer has a valid email address and is not already logged in with another account. **Main Success Scenario:**

1. Customer selects "Create Account" or "Join Loyalty Program" from the website or kiosk. 2. Customer enters personal information (name, email, optional phone) and creates a password. 3. System validates email uniqueness and sends a verification link.
4. Customer verifies email and logs in for the first time.
5. Customer optionally adds payment methods and sets preferences (favorite theaters, genres). 6. System creates the loyalty account and awards initial welcome points.

Extensions:

- Email already registered → System offers password reset or login.
- Customer forgets password → Standard secure password reset flow via email. • Password security → Enforces general password requirements.

UC-03: Admin Override Purchase or Manage Showtimes

Actor(s): Theater Staff / Administrator
Purpose: Allow authorized staff to correct customer purchase errors, adjust showtimes, or manage theater configurations.
Entry Conditions: Staff member has valid admin credentials and appropriate permissions. **Main Success Scenario:**

1. Staff logs into the admin portal using secure credentials.
2. Staff searches for a specific customer purchase or showing.
3. Staff selects the override action (e.g., refund, seat change, showtime adjustment).
4. System validates the change against business rules and records the action in the audit log. 5. System notifies the affected customer (if applicable) and updates all relevant displays and inventory.

Extensions:

- Attempted override violates business rules (e.g., showtime conflict) → System blocks the

action and explains the reason.

Use Case Diagram (Text Representation):
Customer —— Purchase Tickets ———> System
Customer —— Create/Manage Loyalty Account ——> System
Theater Staff —— Admin Override / Manage Showtimes ——> System
(<<includes>> Authenticate for protected actions)

3.5 Non-Functional Requirements

3.5.1 Performance

The system shall support a minimum of 1,000 concurrent users during peak evening hours with average page load times under 3 seconds and checkout completion under 30 seconds for typical transactions.

3.5.2 Reliability & Availability

The system shall maintain 99.5% availability during operating hours. Scheduled maintenance shall be performed during low-traffic periods with advance notice and minimal service interruption.

3.5.3 Security

All sensitive data (payment information, personal data) shall be encrypted in transit (TLS 1.3) and at rest. Tickets shall be generated as unique, non-replicable tokens. Only one concurrent session per user account is permitted.

3.5.4 Usability

The user interface shall be intuitive for users with varying technical skills. New users should be able to complete a ticket purchase with no more than 5 minutes of familiarization. The interface shall be responsive and accessible (WCAG 2.1 AA compliance target).

3.5.5 Maintainability & Portability

The system shall be built with modular, well-documented code to allow low-effort updates. Updates shall be deployable with zero or near-zero downtime. The web application shall run on standard cloud infrastructure and be portable across major cloud providers.

---

4. Software Design Specification

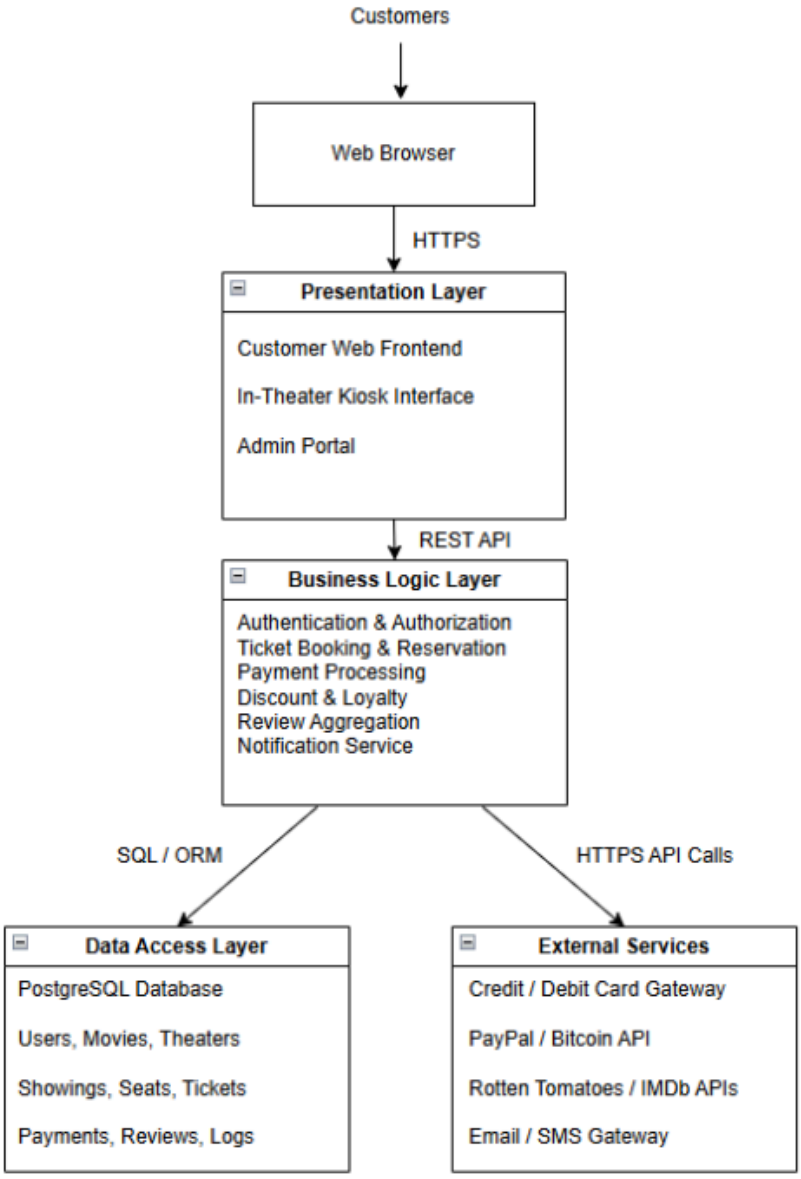
4.1 System Description

The San Diego Movie Theater Ticketing System is a web-based platform that enables customers to browse movies and showtimes across 20 theaters in the San Diego area, select and reserve specific seats (assigned seating in deluxe theaters with 75 seats; open seating in regular theaters with 150 seats), purchase up to 20 tickets per transaction, manage optional loyalty accounts, view aggregated movie reviews and critic quotes from Rotten Tomatoes and IMDb, and receive digital or physical tickets. The system supports English, Spanish, and Swedish interfaces, multiple payment methods (credit/debit cards, PayPal, Bitcoin), discounts for eligible customers, post-purchase feedback, and a 5-minute purchase timer. It includes a secure admin portal for theater staff to manage showtimes, process overrides, view reports, and handle exceptions while maintaining daily transaction logs and unique/non-replicable tickets to prevent scalping. The system is accessible via modern web browsers and in-theater kiosks, designed for high concurrency (minimum 1,000 simultaneous users during peak hours) with real-time seat availability updates.

4.2 Software Architecture Overview

4.2.1 Architectural Diagram of All Major Components

The system follows a layered architecture (Presentation → Business Logic → Data Access) with clear separation of concerns for maintainability and scalability.



Major Components and Connectors:

- Presentation Layer
  - Customer Web Frontend (responsive React/Vue.js-based UI for browsing, seat selection, checkout, loyalty management)
  - Kiosk Interface (simplified touchscreen UI)
  - Admin Portal (staff dashboard for showtime management, overrides, reporting)

- Business Logic Layer (core API layer)
  - Authentication & Authorization Module

- Ticket Booking & Reservation Module (seat map logic, 20-ticket limit, 5-min timer, single-session enforcement)
- Payment Processing Module (integrates with external gateways)
- Discount & Loyalty Module
- Review Aggregation Module (external API calls)
- Notification Module (email/SMS for tickets and feedback)

- Data Access Layer

  - Database (PostgreSQL) – stores Users, LoyaltyAccounts, Movies, Theaters, Showings, Seats, Tickets, Payments, Reviews, Logs

- External Integration Layer

  - Payment Gateways (Stripe/PayPal for cards & PayPal; Bitcoin API)

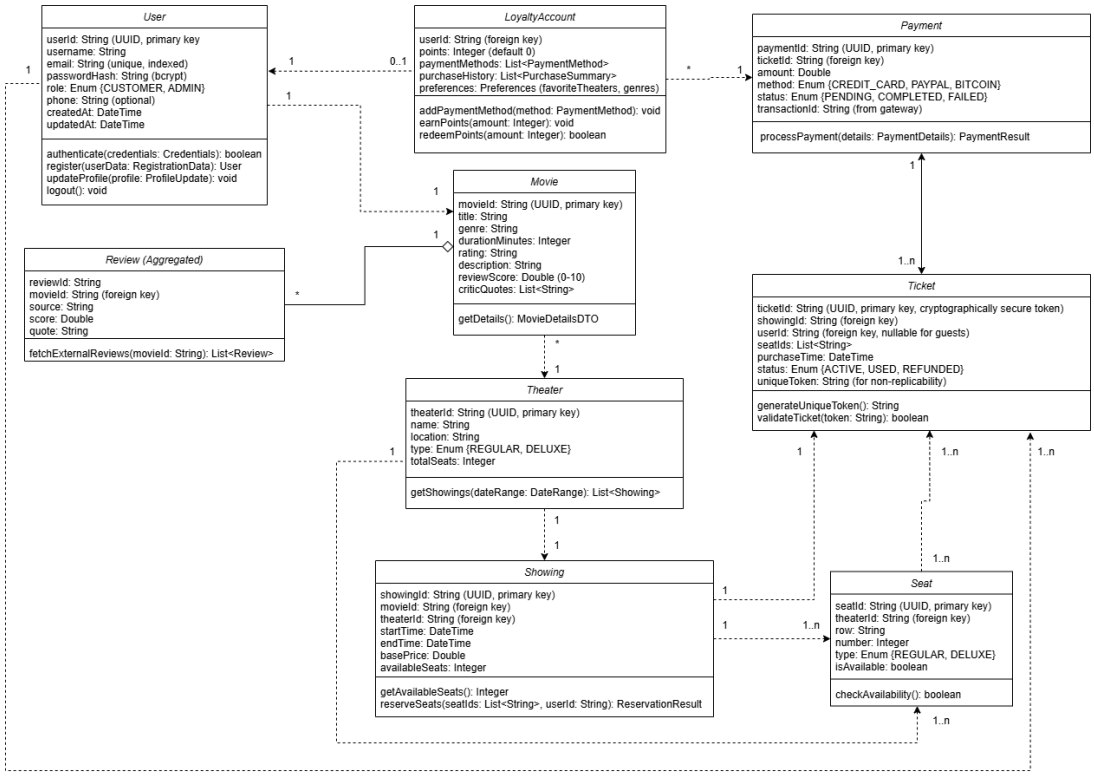
  - Movie Review APIs (Rotten Tomatoes, IMDb)

  - Email/SMS Gateway (e.g., SendGrid/Twilio)

Connectors/Flows:

Frontend ↔ REST/GraphQL API (HTTPS/TLS) ↔ Business Logic Modules ↔ Database (JDBC/ORM) and External Services (HTTPS). Real-time updates via WebSockets for seat availability. All sensitive data encrypted in transit (TLS 1.3) and at rest.

4.2.2 UML Class Diagram



Key Classes (with detailed attributes and operations):

Class: User

Attributes:

- userId: String (UUID, primary key)
- username: String
- email: String (unique, indexed)
- passwordHash: String (bcrypt)
- role: Enum {CUSTOMER, ADMIN}
- phone: String (optional)

- createdAt: DateTime
- updatedAt: DateTime

Operations:
- authenticate(credentials: Credentials): boolean
- register(userData: RegistrationData): User
- updateProfile(profile: ProfileUpdate): void
- logout(): void

Class: LoyaltyAccount (1:1 association with User)
Attributes:
- accountId: String (UUID, primary key)
- userId: String (foreign key)
- points: Integer (default 0)
- paymentMethods: List<PaymentMethod>
- purchaseHistory: List<PurchaseSummary>
- preferences: Preferences (favoriteTheaters, genres)

Operations:
- addPaymentMethod(method: PaymentMethod): void
- earnPoints(amount: Integer): void
- redeemPoints(amount: Integer): boolean

Class: Movie
Attributes:
- movieId: String (UUID, primary key)
- title: String
- genre: String
- durationMinutes: Integer
- rating: String
- description: String

- reviewScore: Double (0-10)
- criticQuotes: List<String>

Operations:
- getDetails(): MovieDetailsDTO

Class: Theater
Attributes:
- theaterId: String (UUID, primary key)
- name: String
- location: String
- type: Enum {REGULAR, DELUXE}
- totalSeats: Integer

Operations:
- getShowings(dateRange: DateRange): List<Showing>

Class: Showing
Attributes:
- showingId: String (UUID, primary key)
- movieId: String (foreign key)
- theaterId: String (foreign key)
- startTime: DateTime
- endTime: DateTime
- basePrice: Double
- availableSeats: Integer

Operations:
- getAvailableSeats(): Integer
- reserveSeats(seatIds: List<String>, userId: String): ReservationResult

Class: Seat

Attributes:

- seatId: String (UUID, primary key)
- theaterId: String (foreign key)
- row: String
- number: Integer
- type: Enum {REGULAR, DELUXE}
- isAvailable: boolean

Operations:

- checkAvailability(): boolean

Class: Ticket

Attributes:

- ticketId: String (UUID, primary key, cryptographically secure token)
- showingId: String (foreign key)
- userId: String (foreign key, nullable for guests)
- seatIds: List<String>
- purchaseTime: DateTime
- status: Enum {ACTIVE, USED, REFUNDED}
- uniqueToken: String (for non-replicability)

Operations:

- generateUniqueToken(): String
- validateTicket(token: String): boolean

Class: Payment

Attributes:

- paymentId: String (UUID, primary key)
- ticketId: String (foreign key)
- amount: Double

- method: Enum {CREDIT_CARD, PAYPAL, BITCOIN}
- status: Enum {PENDING, COMPLETED, FAILED}
- transactionId: String (from gateway)

Operations:
- processPayment(details: PaymentDetails): PaymentResult

Class: Review (aggregated)
Attributes:
- reviewId: String
- movieId: String (foreign key)
- source: String
- score: Double
- quote: String

Operations:
- fetchExternalReviews(movieId: String): List<Review>

Associations (text form for diagram):
User1 ── 0..1LoyaltyAccount
LoyaltyAccount*──1Payment
Payment1──1..nTicket
Ticket1..n──1..nSeat
Seat1..n──1Showing
Showing1──1Theater
Theater1──*Movie
Review*──1Movie
Movie1──1User

Ticket1..n──1User
Seat1..n──1Theater
Ticket1──1Showing
Movie1──1User

4.2.3 Description of Classes, Attributes, and Operations

(See the detailed attribute/operation lists above in 4.2.2. All datatypes are specified. Operations include clear parameter types and return types. For example, `reserveSeats(seatIds: List<String>, userId: String): ReservationResult` where `ReservationResult` is a DTO containing success flag, reserved seats, and error message if any.)

4.3 Development Plan and Timeline

Partitioning of Tasks (8-week prototype timeline)

- Week 1–2: Database design, schema implementation, initial data seeding, and Entity-Relationship modeling.
- Week 2–4: Backend API development (authentication, booking logic, payment integration, discount engine).
- Week 3–5: Frontend development (responsive UI, interactive seat maps, checkout flow, loyalty features).
- Week 4–6: Kiosk interface, external API integrations (reviews + payments), and notification service.
- Week 5–7: Admin portal, reporting/logging features, and security hardening (unique tokens, session management).
- Week 6–8: Comprehensive testing (unit, integration, end-to-end, security), bug fixing, performance optimization, and user acceptance testing.
- Week 7–8: Documentation updates, deployment to staging/production, and final polish.

Team Member Responsibilities

- Anthony Alarcon: Backend development, database design, payment integration, security features, and overall architecture. (Primary owner of Weeks 1–4 backend tasks + security.)
- Eric Phan: Frontend and kiosk UI/UX development, responsive design, seat selection interface, and customer-facing flows. (Primary owner of Weeks 3–6 frontend tasks.)
- Ven Nguyen: Admin portal, reporting & logging, external review API integration, comprehensive testing, and QA. (Primary owner of Weeks 5–8 admin/testing tasks.)

Each team member is required to make at least one commit to the shared GitHub repository. The project uses feature branches and pull requests for collaboration. Estimated completion: fully functional prototype ready for user testing by the end of Week 8.

5. Test Plan

5.1 Testing Strategy Overview

This test plan defines the verification and validation approach for the San Diego Movie Theater Ticketing System. Testing is organized into three granularities:
- **Unit Testing** – Isolated testing of individual classes/modules (Ticket, Seat, Discount engine, etc.)

- **Integration / Functional Testing** – Testing of interacting modules (Booking + Payment + Ticket generation, Loyalty + User, Admin override flows)

- **System Testing** – End-to-end scenarios that exercise the full stack (Presentation → Business Logic → Data Access → External Services), including concurrency and localization

The highest-risk areas identified from the requirements and design are:

1. Unique, non-replicable ticket generation (anti-scalping)

2. Seat inventory consistency under concurrent load (no overselling)

3. Complete purchase flow (seat hold → discount → payment → ticket delivery)

4. 5-minute inactivity timer and session enforcement

5. Admin override + audit logging

6. Multi-language support and alternative payment methods (Bitcoin)

Test cases are mapped to the Functional Requirements (FR-01 … FR-14) and Use Cases (UC-01, UC-02, UC-03) defined earlier in this document.

5.2 Test Levels and Scope

Performance testing verifies that the system can support a minimum of 1,000 concurrent users with an average load time of less than 3 seconds and checkout completion of 30 seconds.

The load tests will simulate heavy traffic during popular movie releases (rush hour) to ensure that the system remains optimized and responsive. Stress tests will be routinely performed to identify the system's max capacity and recovery performance after rush hour settles down.

5.2.1 Unit Level

Focuses on pure logic with no external dependencies (or with mocks). Primary targets:

- `Ticket.generateUniqueToken()` – cryptographic uniqueness

- `Seat.checkAvailability()` – correct availability state

- Discount calculation rules (student / military / senior + weekday restrictions)

5.2.2 Integration / Functional Level

Validates that collaborating modules produce correct outcomes together:

- Full purchase pipeline (seat selection → discount → payment gateway → unique ticket creation)

- Loyalty account registration + email verification + welcome points

- Reservation hold + 5-minute timer release

- Admin refund / seat-change override + audit log entry

5.2.3 System Level

Exercises the complete running system:

- Concurrent last-seat race conditions

- Guest end-to-end purchase on a regular (open-seating) theater including digital delivery

and feedback prompt

- Full flow under Spanish or Swedish UI + Bitcoin payment

5.3 Test Case Summary

Ten concrete test cases are provided in the accompanying Excel file `SDTheaterTickets_TestCases.xlsx`.
They satisfy the minimum requirement of at least two test sets per granularity (actual distribution: 3 Unit / 4 Integration / 3 System).

The Excel file contains the full details (TestCaseId, Component, Priority, Description, Pre-requisites, Test Steps, Expected Result, etc.).

5.3.1 Requirements Traceability

The following table maps out the functional requirements defined in this SRS to the corresponding test cases. This ensures that each major system requirement is covered by one or more tests

Functional Requirement	Related Test Cases
FR-01 Browse Showtimes and Movies	SYS_EndToEnd_Guest_09
FR-02 Select Seats	UNIT_Seat_02, INT_Purchase_04
FR-03 Purchase Tickets	INT_Purchase_04
FR-04 Multiple Payment Methods	INT_Purchase_04, SYS_EndToEnd_Guest_09
FR-05 Non-replicable Tickets	UNIT_Ticket_01
FR-06 Ticket Delivery Options	SYS_EndToEnd_Guest_09
FR-07 Purchase Window	INT_Reservation_06
FR-08 Optional Loyalty Accounts	INT_Loyalty_05
FR-09 Discounts	UNIT_Discount_03
FR-10 Post-Purchase Feedback	SYS_EndToEnd_Guest_09
FR-11 Administrative Functions	INT_AdminOverride_07
FR-12 High-Demand Queueing	SYS_Concurrent_08
FR-13 Daily Purchase Logging	INT_AdminOverride_07
FR-14 Multi-Language Support	SYS_MultiLang_Payment_10

5.4 Remaining / Suggested Expansion Work

- Additional negative / edge-case unit tests (invalid discount codes, malformed seat IDs, expired tokens)

- Performance / load tests against the 1,000 concurrent user requirement

- Security-focused tests (session fixation, privilege escalation on admin portal, ticket token reuse attempts)

- Kiosk-specific system tests (touch UI, printer integration, offline behavior)

- High-demand queue behavior (FR-12)

- More thorough localization and accessibility checks

- Regression suite structure and CI integration notes

5.5 Design Specification Note

The UML Class Diagram and class descriptions from Section 4 (Software Design Specification) remain current and sufficient for this test plan. No structural changes to the class model were required for the test cases defined above as no extra features or benefits were requested.

Minor attribute-level clarifications (e.g., exact token format) may be added later if implementation details change.

5.6 GitHub

Test plan artifacts (this section + the Excel file) will be committed to the group repository:

https://github.com/vnguyen6237/SDTheaterTickets_SRS

6. Architecture Design with Data Management

6.1 Updated Software Architecture Diagram

The architecture remains a layered design (Presentation → Business Logic → Data Access + External Services). For this iteration we refined the Data Access Layer to more clearly show how persistent data is organized and managed. The diagram below reflects the current design with explicit attention to the primary database and its logical groupings.

Architecture Overview (text representation of the updated diagram):

```
Customers / Staff

↓ HTTPS (TLS 1.3)

Presentation Layer
Customer Web Frontend | In-Theater Kiosk | Admin Portal

↓ REST API

Business Logic Layer
Auth | Booking & Reservation | Payment | Discount/Loyalty | Reviews |

Notification ↓ SQL/ORM ↓ HTTPS API Calls

Data Access Layer External Services
PostgreSQL (Primary DB) Payment Gateways
- Users & Auth schema (Stripe / PayPal / Bitcoin)
- Catalog schema (Movies, Theaters) Review APIs (RT / IMDb)
- Inventory schema (Showings, Seats) Email/SMS Gateway
- Transactions schema (Tickets, (SendGrid / Twilio)
- Payments, Logs)
```

Changes from previous design (Assignment 2): The high-level layers are unchanged. We added explicit logical schema groupings inside the PostgreSQL database so the data management strategy is visible in the architecture itself. No new external services or presentation components were introduced.

6.2 Data Management Strategy

6.2.1 Choice of Database Technology

We selected a single primary **PostgreSQL** relational database for all persistent application data. PostgreSQL was chosen because:

- It provides strong ACID guarantees, which are essential for ticket inventory and payment-related operations (we cannot oversell seats or lose payment records).
- It has mature support for concurrent access and row-level locking, which helps with the real-time seat reservation problem.
- JSON/JSONB columns give flexibility for semi-structured data (preferences, critic quotes, payment method metadata) without needing a second store
- It is well-supported in cloud environments and works cleanly with common ORMs (e.g., SQLAlchemy, Prisma, TypeORM).
- The team already has experience with relational modeling, reducing implementation risk for an 8-week prototype.

6.2.2 Number of Databases and Logical Organization

We use **one primary PostgreSQL database** with four logical schemas (or schema-like table groups). This keeps operational complexity low while still separating concerns:

Logical Group	Main Tables	Purpose
Users & Auth	Users, LoyaltyAccounts, Sessions	Identity, credentials, loyalty profiles, session control (single concurrent login)
Catalog	Movies, Theaters, Reviews	Relatively static reference data that changes infrequently
Inventory	Showings, Seats	High-contention data: seat availability and showtime capacity. Most critical for concurrency control
Transactions	Tickets, Payments, PurchaseLogs, AuditLog	Immutable or append-heavy records for purchases, refunds, and auditing

All groups live in the same physical database instance. We do not currently split them across multiple database servers. Foreign keys are used between groups where referential integrity matters (e.g., Ticket → Showing, Ticket → User, Seat → Theater).

6.2.3 Data Sensitivity and Protection

Payment card data is never stored in our database; it is handled by the external payment gateways (Stripe/PayPal). We store only tokens or transaction IDs. Passwords are stored as bcrypt hashes. Personally identifiable information (email, phone, name) is encrypted at rest using database-level or application-level encryption. All connections use TLS 1.3. Daily transaction logs and audit records support both business reporting and security investigation.

6.2.4 Consistency and Concurrency Approach

Seat reservation is the highest-contention operation. We rely on PostgreSQL transactions with appropriate isolation (typically READ COMMITTED with explicit locking or optimistic checks on the availableSeats / isAvailable fields). A short-lived hold (the 5-minute timer) is recorded in the database so that abandoned reservations can be released by a background job or by the timer service itself. Unique ticket tokens are generated with a cryptographically secure source and stored with a uniqueness constraint to prevent duplication.

6.2.5 Backup and Recovery

The PostgreSQL database will be automatically backed up on a regular schedule to ensure that no data has been lost. Incremental backups will be performed daily, while full backups will be performed weekly. Transaction logs will allow recovery to the most recent consistent state in the event of hardware failure or data getting corrupted. Backups will be encrypted and stored securely in a separate location.

6.3 Design Decisions and Tradeoffs

6.3.1 Single PostgreSQL Database vs. Multiple Databases

Chosen approach: One primary PostgreSQL database containing all four logical groups. **Alternatives considered:**

- Separate databases per logical group (Users DB, Inventory DB, Transactions DB, etc.). • Polyglot persistence (e.g., PostgreSQL for transactions + Redis for seat holds + MongoDB for reviews). • Fully managed serverless options (e.g., DynamoDB or Firestore) for the entire data layer.

Tradeoffs of our choice:

Advantages: Simpler operations and deployment for a student team and a prototype timeline. Strong transactional consistency across related tables (Ticket + Payment + Seat update can happen in one transaction). Easier joins for reporting and admin queries. Lower cognitive load and fewer moving parts to secure and back up.

Disadvantages / Risks: A single database can become a scalability bottleneck under extreme load. Schema changes affect the whole system. We cannot independently scale the Inventory group without scaling everything else. If we later need geographic distribution or very different access patterns, we may have to introduce read replicas or split the database.

Why we accepted the tradeoff: The system is scoped to 20 theaters in one metro area and a target of ~1,000 concurrent users. PostgreSQL with proper indexing and connection pooling is expected to handle this comfortably. The prototype schedule favors simplicity and correctness over premature distribution. While the system grows, scalability can be improved by introducing read replicas, database partitioning, or separating high-demand components such as inventory management into their own databases. These changes would help allow the system to support a large number of theaters and users while maintaining good performance and reliability.

6.3.2 Relational Model vs. Document / NoSQL Store

A pure document store (MongoDB, etc.) was considered for flexibility with reviews and preferences. We rejected it as the primary store because seat inventory and payment flows require strong consistency and multi-row transactions. PostgreSQL's JSONB support gives us most of the flexibility we need for semi-structured fields while keeping ACID properties for the critical paths. This is a deliberate tradeoff: slightly more rigid schema in exchange for safer concurrent updates and clearer integrity rules.

6.3.3 In-Database Seat Holds vs. External Cache (Redis)

An alternative design would keep short-lived seat holds only in Redis (or another in-memory store) and write to PostgreSQL only on successful purchase. This can reduce database load and make expiry trivial. We chose to keep holds in PostgreSQL for the prototype so that the source of truth stays in one place and we avoid dual-write consistency problems. If load testing shows the Inventory tables becoming a hotspot, introducing Redis for holds is a natural next step and would not require rewriting the rest of the data model.

6.3.4 Summary of Key Tradeoffs

Decision	Main Benefit	Main Cost / Risk
Single PostgreSQL DB	Simple ops, strong ACID, easy joins	Harder to scale individual parts independently
Logical schemas only (not separate DBs)	Low complexity for team & timeline	Less isolation between data domains
JSONB for flexible fields	Avoids second database for reviews/prefs	Slightly less query optimization than pure columns
Seat holds in PostgreSQL	Single source of truth	Higher write load on Inventory tables under peak

No card data stored locally	Reduced PCI scope & security risk	Dependence on external payment gateways

- **Relation to Previous Design Work**

The class diagram and component list from Assignment 2 remain valid. The only meaningful update for this assignment is the explicit treatment of data organization and the documented rationale for the single-PostgreSQL approach. No classes were added or removed; the data management decisions simply make the persistence strategy visible and justified.

A. Appendices

A.1 Source Requirements Traceability

All functional and non-functional requirements in this document are derived from and traceable to the client interview notes (Theater Ticketing System Qs.rtf), elicited requirements lists, and additional stakeholder input provided in July 2026.

A.2 Future Considerations (Out of Scope for v1)

- Native mobile applications
- Monthly/annual subscription passes
- Expansion beyond San Diego
- Integration with third-party ticket resale platforms
- Advanced analytics dashboard for executives

— End of Software Requirements Specification v1.0 —
https://github.com/vnguyen6237/SDTheaterTickets_SRS